Date: 22nd April 2026 **Project Supervisor:** Prof. Bhaskar Biswas

Analysis of Information Diffusion Models in Social Networks

Rahul Kumar Das

Roll No. 24075102

Department of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi

Exploratory Research Project | Semester IV

Contents

1 Abstract

It is very important to know how information, behaviors, or diseases spread through a network in fields like public health and viral marketing. This exploratory project centers on the execution and comparative evaluation of essential information diffusion models within social network graphs. We simulate and assess both progressive models (Independent Cascade, Linear Threshold) and epidemic compartmental models (SIR, SIS). By looking at how these cascades change over time, this report shows how different diffusion mechanisms are sensitive to changes in structure and parameters. It also sets up a basic framework for future network optimization tasks.

2 Introduction

These processes depend heavily on how the underlying topology is structured and what kind of stochastic behavior takes place in this process of propagation.

- **Purpose of the Project:** The ultimate objective of this project will be to build an efficient simulation environment which allows us to simulate and study all sorts of network diffusion models.
- **Key Points:** We employ various theoretical models, some of which involve permanent activation (IC model) and others which involve transient activation (SIS model). In the end, we describe several models of competitive diffusion.
- **Justification of the Problem:** To address highly complicated optimization problems in networked environments (e.g. identifying key influencers or devising quarantine methods), we first have to simulate them correctly.

3 Network Datasets

To evaluate the diffusion models, experiments were conducted on distinct network topologies, providing a mix of directed and undirected edges, as well as varying degrees of clustering.

- **ego-Facebook (SNAP):** An undirected graph representing dense social circles and community clusters.

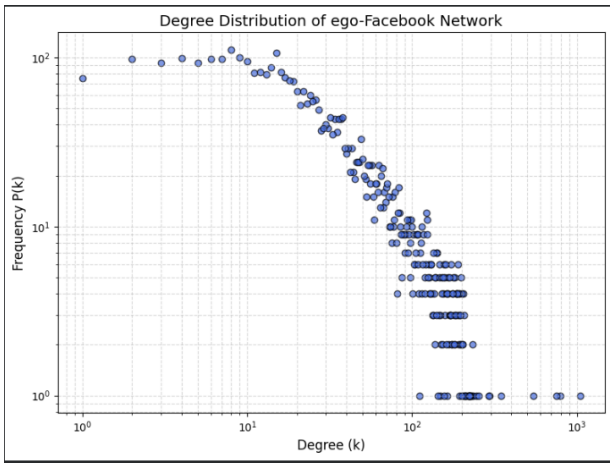


Figure 1: Degree distribution of the ego-Facebook dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 4039, 88234 \rangle$. The heavy-tailed distribution indicates the presence of highly connected "hub" nodes, characteristic of real-world scale-free social networks.

- **wiki-Vote (SNAP):** A directed graph representing administrator voting on Wikipedia, useful for observing hierarchical influence flow.

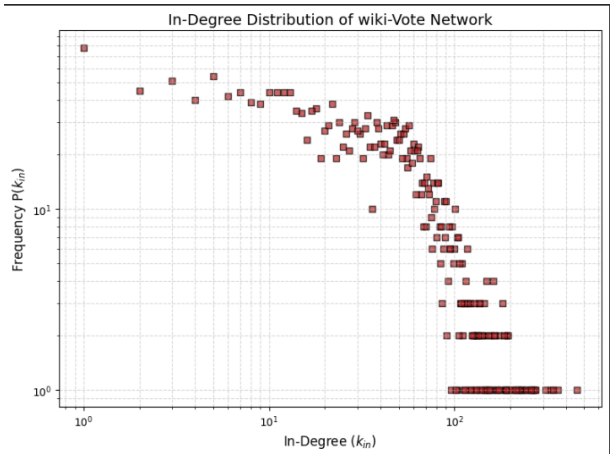


Figure 2: In-degree distribution of the wiki-Vote dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 7115, 103689 \rangle$. As a directed graph, the heavy-tailed in-degree distribution highlights the strict hierarchy of influence and votes received by highly active administrators.

4 Centrality Measures and Seed Selection

The initial nodes chosen to start a cascade (the seed set) heavily dictate the final reach of the diffusion. To establish a baseline for our simulations, various centrality measures were evaluated to rank and select the initial seeds.

- **Degree Centrality:** A radial volume-based measure that is the simplest to compute, as it simply equates to the node degree. It can be computed using the adjacency matrix of the network by summing all length-1 paths starting from a node.

$$C_D(u) = \deg(u) \tag{1}$$

- **Closeness Centrality:** A radial length-based measure that finds the closeness of a node to all other nodes. It is defined as the average of all the shortest paths from a given node to all other nodes.

$$C_C(u) = \frac{\sum_{v \in V \setminus \{u\}} d(u,v)}{|V| - 1} \tag{2}$$

  where $d(u, v)$ is the shortest distance between node u and node v.

- **Node Betweenness Centrality:** A medial centrality measure. It computes the importance of a node based on its existence in the shortest paths of any pair of nodes.

$$C_B(u) = \sum_{\substack{w,v \in V \\ w \neq v \neq u}} \frac{SP_{w,v}(u)}{SP_{w,v}} \tag{3}$$

  where $SP_{w,v}$ is the total number of shortest paths between w and v, and $SP_{w,v}(u)$ is the total number of those shortest paths that pass through u.

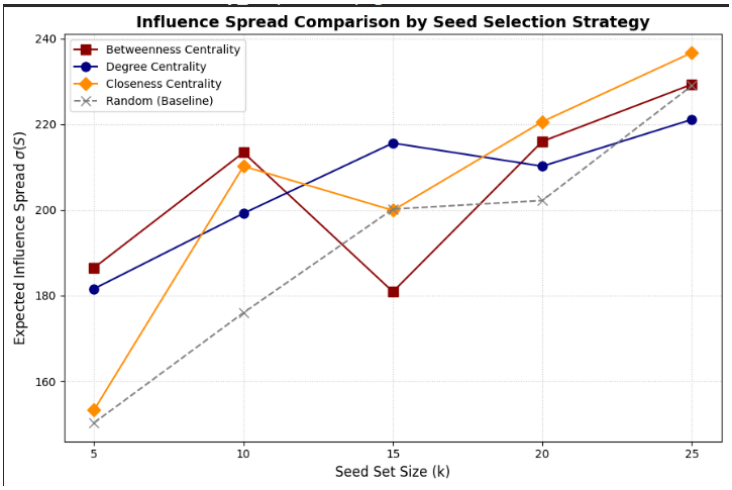


Figure 3: Comparison of information spread across different seed selection strategies.

5 Background and Related Work: Diffusion Models

To accurately simulate network diffusion, several foundational theoretical models must be defined. We categorize these into cascading, threshold, epidemic, and competitive frameworks. The core logic for each model is detailed below.

5.1 Threshold Diffusion Models

Threshold models dictate that node activation is driven by cumulative peer pressure. Every edge is allied with a weight and every node has a threshold value.

- **Linear Threshold Model (LT):** A node y activates only if the sum of the positive influence weights W_{xy} from its already-active neighbors exceeds its internal threshold T_y:

$$\sum_{x \in N^a(y)} W_{xy} \geq T_y \quad \text{where} \quad \sum_{x \in N(y)} W_{xy} \leq 1 \tag{4}$$

```
while newly_active:
        new_activations = set()
        for node in G.nodes():
            if node in active:
                continue
            influence = 0
            for neighbor in G.neighbors(node):
                if neighbor in active and (neighbor, node) in
   weights:
                    influence += weights[(neighbor, node)]
            if influence >= thresholds[node]:
                new_activations.add(node)
        newly_active = new_activations - active
        active.update(newly_active)
```

Listing 1: Core logic for LT Model

- **Majority Threshold Model (MT):** A node adopts a behavior if and only if strictly the majority of its immediate neighbors are active.

$$T_y = \frac{1}{2} d(y) \tag{5}$$

```

while newly_active:
        new_activations = set()
        for node in G.nodes():
            if node in active:
                continue
```

```
            neighbors = list(G.neighbors(node))
            if len(neighbors) == 0:
                continue
            active_neighbors = sum(1 for n in neighbors if n in
    active)
            if active_neighbors / len(neighbors) >= 0.5:
                new_activations.add(node)
        newly_active = new_activations - active
        active.update(newly_active)
```

Listing 2: Core logic for Majority Threshold

- **Small Threshold Model (ST):** Threshold values are small constants (e.g., $T_y = 1$ or $T_y = 2$).

```
while newly_active:
        new_activations = set()
        for node in G.nodes():
            if node in active:
                continue
            influence = 0
            for neighbor in G.neighbors(node):
                if neighbor in active and (neighbor, node) in
    weights:
                    influence += weights[(neighbor, node)]
            if influence >= thresholds[node]:
                new_activations.add(node)
        newly_active = new_activations - active
        active.update(newly_active)
```

Listing 3: Core logic for Small Threshold

- **Unanimous Threshold Model (UT):** The most influence-resistant model where a node requires all of its neighbors to be active.

$$T_y = d(y) \quad \text{for each node } y \in V \tag{6}$$

```
while newly_active:
        new_activations = set()
        for node in G.nodes():
            if node in active:
                continue
            neighbors = list(G.neighbors(node))
            if len(neighbors) == 0:
                continue
            active_neighbors = sum(1 for n in neighbors if n in
    active)
```

```
            if active_neighbors == len(neighbors):
                new_activations.add(node)
        newly_active = new_activations - active
        active.update(newly_active)
```

Listing 4: Core logic for Unanimous Threshold

5.2 Cascading Diffusion Models

Cascading models dictate that propagation occurs through independent probabilistic events along edges, operating in an iterative manner.

- **Independent Cascading Model (IC):** When a node becomes active, it has a single, independent chance to activate each of its currently inactive neighbors with a constant probability $P_y(x)$. Order-independence ensures that the sequence of activation attempts does not affect the final probability:

$$\prod_{i=1}^{k}(1 - P_y(v_i|S \cup M_i)) = \prod_{i=1}^{k}(1 - P_y(v_i|S \cup M_i')) \tag{7}$$

```
while newly_active:
        new_activations = set()
        for node in newly_active:
            for neighbor in G.neighbors(node):
                if neighbor not in active:
                    p = edge_prob[(node, neighbor)]
                    if random.random() < p:
                        new_activations.add(neighbor)
        newly_active = new_activations
        active.update(new_activations)
```

Listing 5: Core logic for Independent Cascade

- **Decreasing Cascading Model (DC):** A more practical variation where the probability of a node successfully activating a neighbor decreases with each subsequent failed attempt. The activation function $P_y(x|S)$ is non-decreasing in S:

$$P_y(x|S) \leq P_y(x|M) \quad \text{for } S \subset M \tag{8}$$

```
while newly_active:
        new_activations = set()
        p = p0 * (decay ** t)
        for node in newly_active:
            for neighbor in G.neighbors(node):
                if neighbor not in active:
```

```
                    if random.random() < p:
                        new_activations.add(neighbor)
        newly_active = new_activations
        active.update(new_activations)
```

Listing 6: Core logic for Decreasing Cascade

- **Generalized Cascade Model (GC):** The activation probability $P_y(x|S) \in [0, 1]$ dynamically depends on the history of the cascade and the set S of already unsuccessfully attempted users.

```
while newly_active:
        new_activations = set()
        for x in newly_active:
            for y in G.neighbors(x):
                if y not in active:
                    S = failed_attempts[y]
                    base_p = prob.get((x, y), 0)
                    p = base_p / (1 + len(S))
                    if random.random() < p:
                        new_activations.add(y)
                    else:
                        S.add(x)
        newly_active = new_activations
        active.update(new_activations)
```

Listing 7: Core logic for Generalized Cascade

5.3 Epidemic Compartmental Models

Originating from biological epidemiology, these models classify populations into distinct states.

- **SIR Model:** Individuals transition through Susceptible → Infectious → Recovered states. Defined by contact rate β and recovery rate γ:

$$\frac{dS}{dt} = -\beta SI, \quad \frac{dI}{dt} = \beta SI - \gamma I, \quad \frac{dR}{dt} = \gamma I \tag{9}$$

```

        for node in G.nodes():
            if state[node] == "I":
                for neighbor in G.neighbors(node):
                    if state[neighbor] == "S":
                        if random.random() < beta:
                            new_state[neighbor] = "I"
                if random.random() < gamma:
```

```
                    new_state[node] = "R"
        state = new_state
```

Listing 8: Node-level transition logic for SIR

- **SIS Model:** People recover with zero immunity, returning immediately to the susceptible state ($S \to I \to S$).

$$\frac{dS}{dt} = -\beta SI + \gamma I, \quad \frac{dI}{dt} = \beta SI - \gamma I \tag{10}$$

```

        for node in G.nodes():
            if state[node] == "I":
                # Infect neighbors
                for neighbor in G.neighbors(node):
                    if state[neighbor] == "S":
                        if random.random() < beta:
                            new_state[neighbor] = "I"
                # Recovery (back to susceptible)
                if random.random() < gamma:
                    new_state[node] = "S"
        state = new_state
```

Listing 9: Node-level transition logic for SIS

- **SIRS Model:** Recovered nodes experience temporary immunity before returning to the susceptible pool due to a loss of immunity rate f.

$$\frac{dS}{dt} = -\beta SI + fR, \quad \frac{dI}{dt} = \beta SI - \gamma I, \quad \frac{dR}{dt} = \gamma I - fR \tag{11}$$

```
for node in G.nodes():
            # Infection
            if state[node] == "I":
                for neighbor in G.neighbors(node):
                    if state[neighbor] == "S":
                        if random.random() < beta:
                            new_state[neighbor] = "I"
                # Recovery
                if random.random() < gamma:
                    new_state[node] = "R"
            # Immunity loss
            elif state[node] == "R":
                if random.random() < delta:
                    new_state[node] = "S"
```

```
        state = new_state
```

Listing 10: Node-level transition logic for SIRS

5.4 Competitive Influence Diffusion Models

Real-world scenarios often involve multiple cascades competing within the network simultaneously.

- **Distance-Based Competitive Model:** In adversarial network scenarios, a node's likelihood of adopting a specific behavior relies heavily on its shortest-path proximity to the competing influential sources. Let I_A and I_B denote the initial seed sets for Cascade A and B, respectively, with $I = I_A \cup I_B$. The expected number of nodes adopting Cascade A is defined as:

$$\rho(I_A|I_B) = \mathbb{E}\left[\sum_{y \in V} \frac{V_y(I_A, d_y(I, E_a))}{V_y(I_A, d_y(I, E_a)) + V_y(I_B, d_y(I, E_a))}\right] \tag{12}$$

  where $d_y(I, E_a)$ represents the smallest distance from node y to the seed set I via active edges E_a, and V_y is the influence valuation based on that distance. The core objective of the influence maximization problem here is to find an optimal seed set I_A of size n that maximizes this expected spread against a fixed opponent I_B:

$$\max\left[\rho(I_A|I_B) : I_A \subseteq (V \setminus I_B), |I_A| = n\right] \tag{13}$$

```
for seed in seeds_A:
        lengths = nx.single_source_shortest_path_length(G, seed)
        for node, dist in lengths.items():
            influence_A[node] = max(influence_A[node], math.exp(-
    alpha*dist))
    # Influence from campaign B
    for seed in seeds_B:
        lengths = nx.single_source_shortest_path_length(G, seed)
        for node, dist in lengths.items():
            influence_B[node] = max(influence_B[node], math.exp(-
    alpha*dist))
    adopted_A, adopted_B, neutral = [], [], []
    for node in G.nodes():
        if influence_A[node] > influence_B[node] and influence_A[
    node] >= threshold:
            adopted_A.append(node)
        elif influence_B[node] > influence_A[node] and influence_B[
    node] >= threshold:
            adopted_B.append(node)
```

```
        else:
            neutral.append(node)
```

Listing 11: Dynamic adoption probability evaluation in the Distance-Based Model

- **Wave Propagation Model:** In this model, influence propagates in discrete temporal steps ($t = 1, 2, \dots$). A node x adopts a behavior at step t strictly if its shortest network distance to the initially active set is less than or equal to t. The expected adoption spread for Cascade A, given the simultaneous presence of Cascade B and active edges E_a, is defined by the probability $P(x)$:

$$\pi(I_A|I_B) = \mathbb{E}\left[\sum_{x \in V} P(x|(I_A, I_B, E_a))\right] \tag{14}$$

Similar to the distance-based approach, the objective is to identify an optimal seed set I_A of size n that maximizes this expected spread against a fixed opposing set I_B:

$$\max\left[\pi(I_A|I_B) : I_A \subseteq (V \setminus I_B), |I_A| = n\right] \tag{15}$$

```
# Campaign A
        for node in wave_nodes_A:
            for neighbor in G.neighbors(node):
                if neighbor not in active_A and neighbor not in
    active_B:
                    potential_influence[neighbor] =
    potential_influence.get(neighbor, {})
                    potential_influence[neighbor]['A'] = decay ** (
    distance_A[node] + 1)
        # Campaign B
        for node in wave_nodes_B:
            for neighbor in G.neighbors(node):
                if neighbor not in active_A and neighbor not in
    active_B:
                    potential_influence[neighbor] =
    potential_influence.get(neighbor, {})
                    potential_influence[neighbor]['B'] = decay ** (
    distance_B[node] + 1)
        for neighbor, inf_dict in potential_influence.items():
            inf_A = inf_dict.get('A',0)
            inf_B = inf_dict.get('B',0)
            if inf_A > inf_B:
                new_wave_A.add(neighbor)
                distance_A[neighbor] = wave + 1
            elif inf_B > inf_A:
                new_wave_B.add(neighbor)
                distance_B[neighbor] = wave + 1
```

```
            else:
                # Tie: assign randomly
                if random.random() < 0.5:
                    new_wave_A.add(neighbor)
                    distance_A[neighbor] = wave + 1
                else:
                    new_wave_B.add(neighbor)
                    distance_B[neighbor] = wave + 1
```

Listing 12: Discrete temporal step evaluation in the Wave Propagation Model

- **Weight-Proportional Threshold (WPT) Model:** Designed to simulate competing products in a shared market, this model resolves simultaneous influence using proportional edge weights. Let ϕ_t represent the total active nodes at step t, partitioned into A-active (ϕ_t^A) and B-active (ϕ_t^B) sets. An inactive node y is triggered to activate if the cumulative weight from all active neighbors exceeds its internal threshold T_y: $\sum_{x \in \phi_t} w_{x,y} \geq T_y$.

  Upon triggering, the probability that node y specifically adopts Cascade A is directly proportional to the fraction of influence exerted by A-active neighbors:

$$P(y \in \phi_t^A \mid y \in \phi_t \setminus \phi_{t-1}) = \frac{\sum_{x \in \phi_t^A} w_{x,y}}{\sum_{x \in \phi_t} w_{x,y}} \tag{16}$$

  If this probability check fails, node y adopts Cascade B.

```
for neighbor in G.neighbors(node):
                w = G[node][neighbor]['weight']
                total_weight += w
                if neighbor in active_A:
                    influence_A += w
                if neighbor in active_B:
                    influence_B += w
            if total_weight == 0:
                continue
            normalized_A = influence_A / total_weight
            normalized_B = influence_B / total_weight
            if normalized_A >= thresholds[node] and normalized_A >
   normalized_B:
                new_active_A.add(node)
            elif normalized_B >= thresholds[node] and normalized_B
   > normalized_A:
                new_active_B.add(node)
```

Listing 13: Weight-Proportional evaluation and probabilistic adoption logic

- **Separate Threshold (Asymmetric) Model:** Unlike models that assume uniform resistance to all contagions, this model accounts for the inherent asymmetric virality

of competing behaviors. It assigns distinct adoption thresholds and specific edge weights for each cascade. Let an inactive node y possess independent thresholds T_y^A and T_y^B, along with corresponding incoming edge weights $w_{x,y}^A$ and $w_{x,y}^B$.

At time step t, node y evaluates the influence of its active neighbors $N(y)$ from the previous step (ϕ_{t-1}). Node y transitions to an A-active state if it satisfies:

$$\sum_{x \in N(y) \cap \phi_{t-1}^A} w_{x,y}^A \geq T_y^A \tag{17}$$

Similarly, it transitions to a B-active state if the opposing influence meets its specific B-threshold:

$$\sum_{x \in N(y) \cap \phi_{t-1}^B} w_{x,y}^B \geq T_y^B \tag{18}$$

(Note: If both conditions are satisfied simultaneously in the same time step, adoption is resolved probabilistically based on the ratio by which each threshold is exceeded).

```
activate_A = influence_A >= self.threshold_A[y]
activate_B = influence_B >= self.threshold_B[y]
if activate_A and not activate_B:
    new_states[y] = 1
elif activate_B and not activate_A:
    new_states[y] = 2
elif activate_A and activate_B:
    if random.random() < 0.5:
        new_states[y] = 1
    else:
        new_states[y] = 2
```

Listing 14: Asymmetric condition evaluation in the Separate Threshold Model

6 Methodology

With the theoretical diffusion models established, this section details the algorithmic implementation, data structures, and execution pipeline used to simulate these cascades. The simulations were programmed in Python, utilizing a modular, object-oriented architecture to seamlessly interchange network topologies, seed selection strategies, and diffusion rules.

6.1 Graph Representation and Parameter Initialization

Networks are represented using adjacency lists via the `NetworkX` library. To ensure the models function mathematically, edge weights and node thresholds are initialized dynam-

ically based on the network topology prior to execution.

- **Cascading & Epidemic Parameters:** For models relying on independent probabilities (IC, SIR, SIS), a global transmission probability parameter (p or β) is applied uniformly across all valid edges during execution.
- **Threshold Parameters:** For the Linear Threshold (LT) model, to satisfy the constraint $\sum_{x \in N(y)} W_{xy} \leq 1$, the incoming edge weights for a node are initialized as $1/\deg(v)$. The internal thresholds θ_v are assigned by sampling from a uniform random distribution $\mathcal{U}(0.01, 0.05)$, representing a network where users are relatively susceptible to early adopters.

```python
def initialize_weights(G):
    weights = {}
    for node in G.nodes():
        neighbors = list(G.neighbors(node))
        if len(neighbors) == 0:
            continue
        weight_value = 1 / len(neighbors)
        for neighbor in neighbors:
            weights[(neighbor, node)] = weight_value
    return weights
```

Listing 15: Dynamic Parameter Initialization for the LT Model

6.2 Seed Selection Architecture

Before a diffusion simulation begins, an initial set of active nodes, the seed set S of size k, must be determined. A dedicated `Seed_Selection` class was implemented to pre-compute and extract these nodes efficiently.

- **Random Selection:** A baseline approach where k nodes are selected via a uniform random distribution from the vertex set V.
- **Heuristic Centrality Selection:** Nodes are ranked according to specific structural metrics (Degree, Betweenness, and Closeness Centrality). To optimize performance on large graphs, the module utilizes a priority queue data structure (`heapq.nlargest`) to extract the top k highest-scoring nodes in $\mathcal{O}(N \log k)$ time, bypassing the need for a full $\mathcal{O}(N \log N)$ array sort.

```python
def degree_based(self):
    top_k = heapq.nlargest(
        self.seed_size, self.degree_dict, key=self.degree_dict.get
    )
    return set(top_k)
```

Listing 16: Heuristic seed selection utilizing a priority queue for computational efficiency

- **Competitive Seed Allocation:** For multi-cascade adversarial environments, the architecture generates mutually exclusive seed sets (S_A and S_B). This ensures that a targeted influence campaign (e.g., seeded via centrality heuristics) can be evaluated simultaneously against a baseline contagion (e.g., seeded randomly) without any overlapping initial nodes.

```
def seed_strategy(G):
    nodes = list(G.nodes())
    random_seeds_A = random.sample(nodes, 10)
    random_seeds_B = random.sample(nodes, 10)

    degree_nodes = sorted(G.degree, key=lambda x:x[1], reverse=True)
    high_degree_nodes = [n for n,d in degree_nodes[:20]]
    high_degree_A = [high_degree_nodes[i] for i in range(0, len(
    high_degree_nodes), 2)]
    high_degree_B = [high_degree_nodes[i] for i in range(1, len(
    high_degree_nodes), 2)]  # avoid overlap

    spreads = []
    for seeds_A, seeds_B in [(random_seeds_A, random_seeds_B), (
    high_degree_A, high_degree_B)]:
        active_A, active_B, neutral, spread_A, spread_B, wave_sizes_A,
    wave_sizes_B = competitive_wave_propagation_influence(G, seeds_A,
    seeds_B)
        spreads.append((len(active_A), len(active_B)))
```

Listing 17: Generating mutually exclusive seed sets for competitive diffusion

6.3 Simulation Mechanics and Monte Carlo Execution

The core execution engine runs in discrete time steps ($t = 0, 1, 2, ...$). Because the diffusion models rely heavily on stochastic processes, a single run does not accurately represent the expected influence spread $\sigma(S)$.

To account for this variance, the system employs orchestration classes (`Experiment` and `SimulationRunner`) to manage Monte Carlo executions. Both follow a strict execution pipeline:

1. **Iterative Execution:** For each tested seed set S, the simulation is executed for R runs (e.g., $R = 20$). The random seed is iteratively incremented for each run to ensure distinct stochastic branching while maintaining experimental reproducibility.
2. **State Tracking:** The simulation steps forward until the queue of newly active nodes is empty, tracking the absolute spread size, total diffusion time, and step-by-step activation history.

3. **Statistical Aggregation:** Once all runs are complete, the engine aggregates the data to output the mean final spread μ and the standard deviation σ.

```python
def monte_carlo_simulation(G, seeds, weights, runs=10):
    spreads = []
    curves = []
    for _ in range(runs):
        thresholds = generate_thresholds(G)
        active, spread = linear_threshold(G, seeds, thresholds, weights)
        spreads.append(len(active))
        curves.append(spread)
    return np.mean(spreads), curves
```

Listing 18: Monte Carlo execution loop ensuring independent stochastic branching

7 Results and Evaluation

This section presents the comparative analysis of information propagation under the different diffusion models and evaluates the efficacy of the implemented seed selection strategies.

7.1 Temporal Spread Dynamics

To evaluate how quickly cascades reach their peak, average prevalence curves were plotted over time for both the progressive and epidemic models.

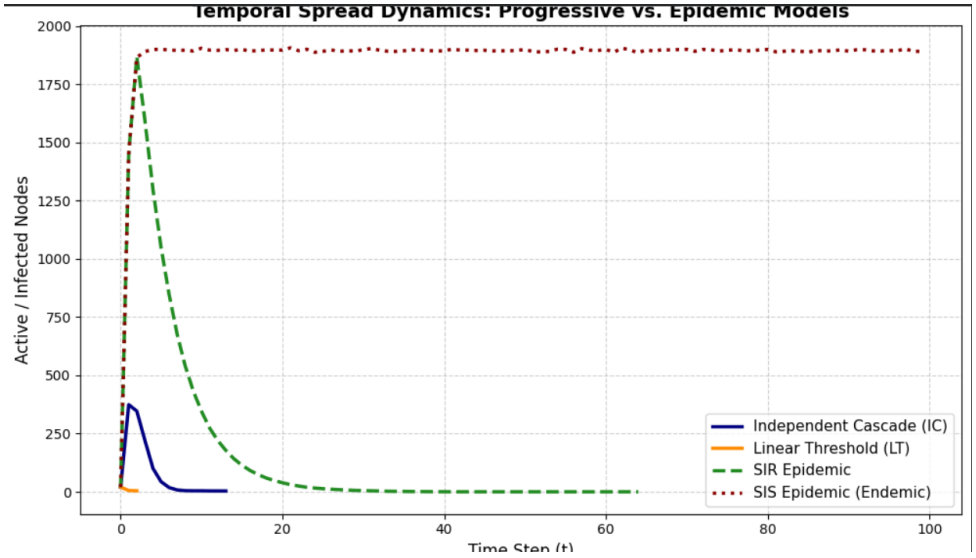


Figure 4: Average prevalence over time comparing progressive models (IC, LT) against epidemic models (SIR, SIS) under a uniform starting seed set ($k = 20$).

The simulations successfully captured the theoretical behaviors of the distinct models. As illustrated in Figure 4, epidemic models reach a distinct inflection point before decaying due to the recovery rate γ (with SIS reaching an endemic steady-state), whereas cas-

cading and threshold models strictly monotonically increase until the cascade naturally terminates when no new edges can be successfully traversed.

7.2 Detailed Metric Breakdown: Seed Strategy vs. Diffusion Model

A critical component of Influence Maximization is understanding the impact of initial seed placement. To isolate the performance differences across the 10 distinct diffusion environments, the following tables break down each core metric individually. Data was aggregated over 20 independent Monte Carlo runs on the ego-Facebook dataset as well as the wiki-Vote dataset using a seed set size of $k = 20$.

7.2.1 Expected Influence Spread (μ)

The primary objective of any viral marketing campaign is to maximize the absolute number of activated nodes. As shown in Table 1 and Table 2, highly restrictive models like the Unanimous Threshold (UT) and Majority Threshold (MT) severely bottlenecked propagation regardless of the seed strategy. Conversely, Betweenness Centrality consistently yielded the highest expected spread in highly contagious environments, as it successfully identifies "bridge" nodes that connect disparate communities.

Table 1: Expected Influence Spread (μ) for Wiki-Vote dataset.

Seed Strategy	**Cascading Models**			**Threshold Models**				**Epidemic Models**		
	IC	**DC**	**GC**	**LT**	**MT**	**ST**	**UT**	**SIR**	**SIRS**	**SIS**
Random (Baseline)	1024.0	359.5	29.6	2215.4	2247.0	105.2	20.0	2304.6	1689.6	1669.0
Degree Centrality	997.9	450.6	199.0	2317.6	2241.0	605.4	24.0	2300.1	1685.9	1649.5
Closeness Centrality	1043.0	342.3	73.7	2307.6	2235.0	500.1	20.0	2290.4	1713,7	1665.9
Betweenness Centrality	1041.3	497.0	141.2	2307.6	2235.0	630.8	21.0	2293.3	1712.1	1661.6

Table 2: Expected Influence Spread (μ) for Facebook dataset.

Seed Strategy	**Cascading Models**			**Threshold Models**				**Epidemic Models**		
	IC	**DC**	**GC**	**LT**	**MT**	**ST**	**UT**	**SIR**	**SIRS**	**SIS**
Random (Baseline)	2114.1	217.4	1669.1	1849.1	95.4	20.0	12.5	4009.8	3075.6	2975.2
Degree Centrality	2110.2	256.0	1814.5	3745.7	510.8	73.0	85.3	4009.6	3129.1	3015.5
Closeness Centrality	2094.3	231.4	1262.3	3992.6	420.5	51.0	65.4	4007.9	3026.9	2992.1
Betweenness Centrality	2112.4	261.5	1861.4	3226.7	540.2	3932.0	98.0	3076.8	1280.6	3011.3

7.2.2 Computational Efficiency (Runtime)

While Influence Maximization prioritizes spread, the algorithmic complexity of identifying seeds is a critical bottleneck for real-world scaling. Table 3 and Table 4 highlight a severe computational trade-off: Degree Centrality achieved near-optimal spread across all 10 models in a fraction of the runtime required for global Betweenness calculations, confirming it as a highly efficient heuristic for large-scale networks.

Table 3: Execution Runtime (in seconds) for each model for Wiki-Vote dataset.

Seed Strategy	Cascading Models			Threshold Models				Epidemic Models		
	IC	DC	GC	LT	MT	ST	UT	SIR	SIRS	SIS
Random	0.12	0.13	0.36	0.35	0.12	0.15	0.05	0.23	1.24	6.31
Degree Centrality	0.16	0.16	0.59	0.34	0.16	0.15	0.07	0.23	1.31	4.98
Closeness Centrality	0.17	0.12	0.45	0.38	0.13	0.30	0.06	0.24	1.40	5.56
Betweenness Centrality	12.6	13.5	13.6	20.5	12.4	12.45	12.7	27.2	13.1	20.90

Table 4: Execution Runtime (in seconds) for each model for Facebook dataset.

Seed Strategy	Cascading Models			Threshold Models				Epidemic Models		
	IC	DC	GC	LT	MT	ST	UT	SIR	SIRS	SIS
Random	0.12	0.05	0.75	0.82	0.04	0.20	0.03	0.64	3.35	16.23
Degree Centrality	0.41	0.09	1.05	1.27	0.06	0.19	0.04	0.61	5.28	16.19
Closeness Centrality	0.40	0.06	0.83	1.25	0.04	0.19	0.04	0.65	3.36	16.98
Betweenness Centrality	10.38	12.08	21.80	14.28	12.05	13.23	12.04	11.61	13.40	6.31

7.2.3 Time-to-Convergence (T)

Convergence time dictates how long a cascade takes to fully saturate the network. Table 5 and Table 6 demonstrate that targeted heuristic seeds generally increase the lifespan of a cascade compared to random seeds, as the diffusion is able to successfully penetrate deeper into the network topology before natural extinction.

Table 5: Time-to-Convergence (T), representing the number of discrete time steps required to naturally terminate or reach an endemic steady-state. (Wiki-Vote dataset)

Seed Strategy	Cascading Models			Threshold Models				Epidemic Models		
	IC	DC	GC	LT	MT	ST	UT	SIR	SIRS	SIS
Random	14.6	8.2	2.8	13.2	3.5	1.0	1.0	40.2	101.0	101.0
Degree Centrality	14.1	3.6	4.4	6.1	5.1	2.0	2.0	44.2	101.0	101.0
Closeness Centrality	14.5	6.2	3.4	7.7	4.5	2.0	1.0	41.0	101.0	101.0
Betweenness Centrality	14.8	3.8	3.9	5.5	2.0	4.0	2.0	42.0	101.0	101.0

Table 6: Time-to-Convergence (T), representing the number of discrete time steps required to naturally terminate or reach an endemic steady-state. (Facebook dataset)

Seed Strategy	Cascading Models			Threshold Models				Epidemic Models		
	IC	DC	GC	LT	MT	ST	UT	SIR	SIRS	SIS
Random	16.1	6.6	14.7	12.9	2.0	12.0	1.0	42.3	101.0	101.0
Degree Centrality	15.5	3.6	14.0	6.7	3.0	10.0	2.0	41.0	101.0	101.0
Closeness Centrality	16.8	4.7	13.0	9.4	3.0	10.0	2.0	45.0	101.0	101.0
Betweenness Centrality	15.5	4.2	12.5	6.4	4.0	9.0	2.0	43.4	101.0	101.0

7.2.4 Cascade Stability and Variance (σ)

Because independent cascade and epidemic models rely on probabilistic edge traversal, analyzing the standard deviation across Monte Carlo runs is critical for assessing risk. As detailed in Table 7 and Table 8, heuristic centrality measures not only increase the

total spread but drastically reduce variance, providing a highly predictable outcome for simulated influence campaigns.

Table 7: Variance (σ) of the final spread across Monte Carlo runs, indicating cascade stability. (Wiki-Vote dataset)

Seed Strategy	**Cascading Models**			**Threshold Models**				**Epidemic Models**		
	IC	**DC**	**GC**	**LT**	**MT**	**ST**	**UT**	**SIR**	**SIRS**	**SIS**
Random	76.8	31.9	19.1	481.3	0.0	0.0	0.0	4.9	40.6	21.4
Degree Centrality	229.7	18.3	5.5	2.1	0.0	0.0	0.0	5.2	43.7	17.5
Closeness Centrality	48.2	50.7	9.0	2.2	0.0	0.0	0.0	4.6	47.6	17.2
Betweenness Centrality	59.2	18.3	4.5	2.1	0.0	0.0	0.0	3.7	30.2	17.3

Table 8: Variance (σ) of the final spread across Monte Carlo runs, indicating cascade stability. (Facebook dataset)

Seed Strategy	**Cascading Models**			**Threshold Models**				**Epidemic Models**		
	IC	**DC**	**GC**	**LT**	**MT**	**ST**	**UT**	**SIR**	**SIRS**	**SIS**
Random	78.8	26.9	19.1	85.3	0.0	0.0	0.0	12.9	74.0	15.7
Degree Centrality	100.9	19.0	5.5	39.2	0.0	0.0	0.0	8.3	73.3	19,4
Closeness Centrality	65.4	21.1	9.0	20.6	0.0	0.0	0.0	9.9	57.4	22.6
Betweenness Centrality	79.0	15.2	4.5	11.1	0.0	0.0	0.0	14.6	63.0	18.0

7.2.5 Impact of Seed Set Size

Beyond the selection strategy itself, the size of the initial seed set k was evaluated to observe scaling behaviors.

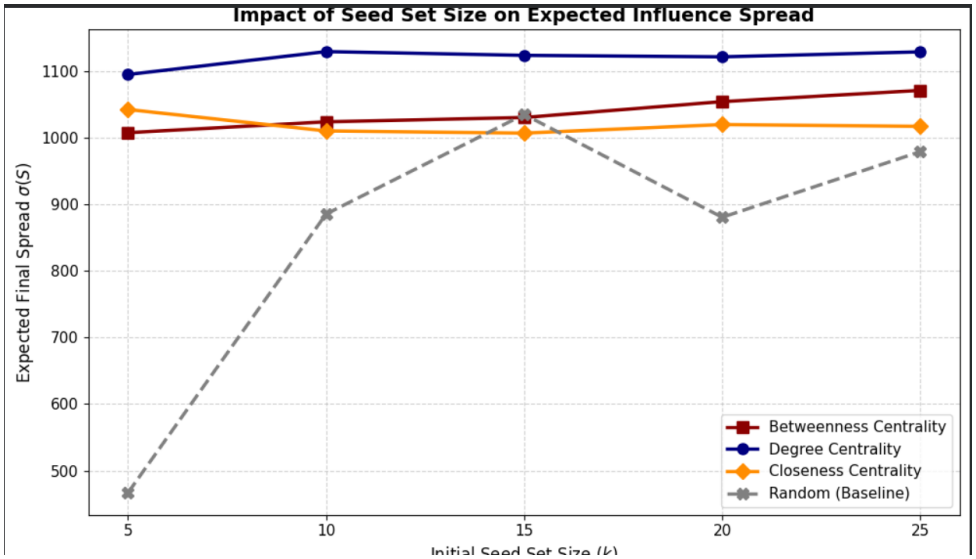


Figure 5: Expected influence spread $\sigma(S)$ evaluated across varying seed set sizes on wiki-Vote Dataset ($k \in \{5, 10, 15, 20, 25\}$).

Evaluating seed set sizes up to $k = 25$ (Figure 5) revealed strict diminishing marginal returns on influence spread across both cascading and threshold families. This behavior empirically supports the submodular nature of the diffusion objective function as mathematically proved by Kempe *et al.*

7.3 Multi-Cascade Competitive Dynamics

To simulate adversarial scenarios—such as competing product launches or rival political campaigns—four distinct multi-cascade competitive models were evaluated. In these simulations, Cascade A (targeted influence seeded via Degree Centrality hubs) competed simultaneously against Cascade B (baseline contagion seeded Randomly), both starting with $k = 20$.

Table 9: Final network adoption states across competitive diffusion models on the ego-Facebook dataset.

Competitive Model	**Cascade A Spread (μ)**	**Cascade B Spread (μ)**	**Neutral**
Distance-Based	2690	1349	0
Wave Propagation	3381	658	0
Weight-Proportional (WPT)	932	43	3064
Separate Threshold (Asym)	3373	666	0

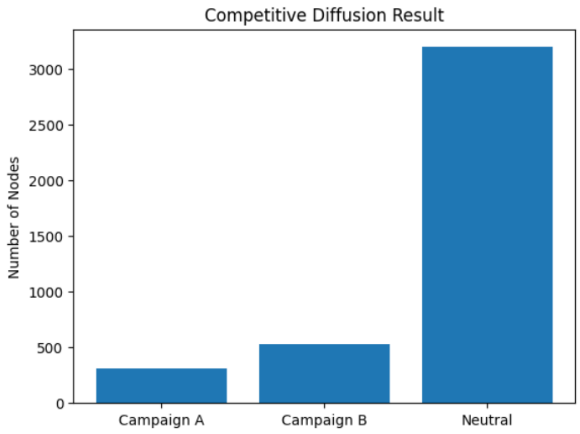
As shown in Table 9, Cascade A consistently dominated the network spread across all four evaluated environments. To further dissect the mechanics of this dominance, unique behavioral metrics were extracted from the temporal progression of each model (Figure 6).

7.3.1 Spatial Blockades and Temporal Acceleration

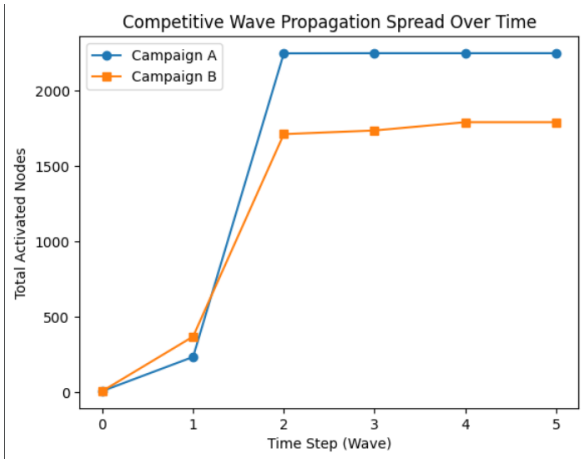
When using distance-centric simulations, the protection of structural hubs ensures a strict topological advantage. In the Distance-Based model (see Figure 6a), the network adoption state graphically shows how Cascade A managed to establish monopoly in the highly connected network core, thereby leaving Cascade B only with fragmented and peripheral clusters. Such spatial isolation results from temporal acceleration, which can be seen in the Wave Propagation model (see Figure 6b) by the sharp increase in wavefront intensity in the initial time steps. The seeds of Cascade B were randomly placed; therefore, when Cascade B got momentum, Cascade A had already depleted all available susceptible nodes, leading to an earlier decline of the wavefront intensity of Cascade B.

7.3.2 Temporal Velocity and Wavefront Intensity

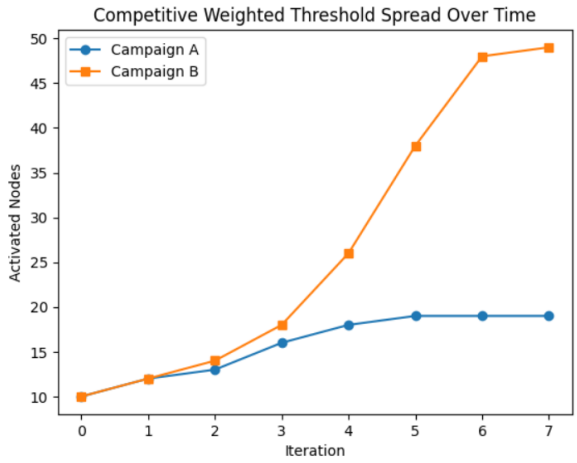
When we use distance-based simulations, making sure that structural hubs are secure gives you a strict time advantage. In the Distance-Based model (see Figure 6a), Cascade A showed a fast adoption rate in the lowest distance tiers ($d = 1, d = 2$) and cut off the network core. You can see this trend in the Wave Propagation model (see Figure 6b) by looking at how quickly the temporal velocity rises at the beginning of the time steps. Cascade A used up all the vulnerable nodes before Cascade B's randomly chosen seeds could start spreading, which is why Cascade B's wavefront fell earlier.



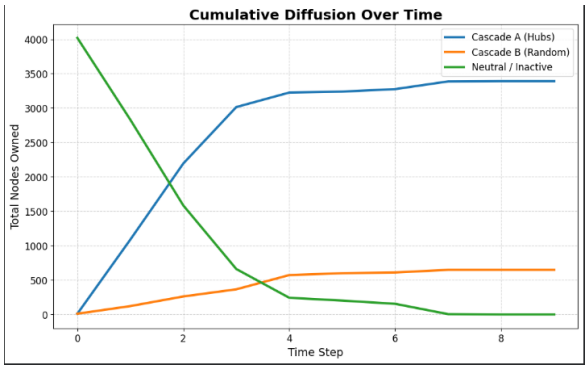
(a) Final Network Adoption State (Distance-Based)



(b) Wavefront Intensity (Wave Prop.)



(c) Accidental Assistance (WPT)



(d) Cumulative Spread (Separate Threshold)

Figure 6: Behavioral analysis of the four competitive diffusion models, highlighting the spatial, temporal, and mechanical advantages of targeted hub-seeding (Cascade A) over random seeding (Cascade B).

7.3.3 Accidental Assistance in Threshold Models

The Weight-Proportional Threshold (WPT) model revealed complex adoption dynamics based on cumulative peer pressure. Tracking the exact triggering mechanisms (Figure 6c) revealed the phenomenon of "accidental assistance." A significant portion of the network was activated not by pure exposure to a single cascade, but by the combined threshold weight of both A and B neighbors. In dense clusters, Cascade B inadvertently helped push stubborn nodes past their activation thresholds, at which point Cascade A successfully won the probabilistic tug-of-war for their final loyalty.

7.3.4 Asymmetric Virality and Cumulative Spread

The Separate Threshold model created an uneven situation in which Cascade B had lower internal thresholds, which meant that it was more likely to spread naturally. Even though this is a natural advantage, tracking the cumulative spread over time (Figure 6d) shows

that Cascade A clearly overtakes Cascade B. This mathematically demonstrates that optimal, centrality-based seed placement can effectively suppress and surpass a competing contagion with a higher natural transmission probability.

8 Discussion and Qualitative Analysis

While the quantitative metrics highlight the absolute performance of the evaluated algorithms, analyzing these results in the context of network topology provides deeper insights into the mechanics of information diffusion.

8.1 Dataset-Topology and Model Interaction Analysis

8.1.1 Ego-Facebook Network Performance (Dense, Undirected Clustering)

There were a large number of clusters on Facebook, and communities that overlapped and clustered very closely together.

- **Cascade Models (IC, DC, GC):** Cascade models did not perform well in this scenario because of the presence of overlapping neighborhoods. These cascade models were victims of the "echo chamber" effect, where all the nodes of the cluster become active but no path connects this cluster to another cluster. This is one of the reasons why Betweenness Centrality outperformed Degree Centrality.
- **Threshold Models (LT, MT, ST, UT):** The graph being dense was not a good trait when it came to the stricter threshold models. The Majority Threshold (MT) and Unanimous Threshold (UT) faced cascade death. However, LT was able to do quite well in this setting, with the help of overlapping triangles and rapid peer pressure accumulation.
- **Epidemic Models (SIR, SIS, SIRS):** Epidemics proved to be very robust to infection since the graphs were undirected and hence, the triangles kept infecting themselves again in the SIS case, leading to an endemic equilibrium. To add immunity temporarily to the equation, the SIRS model was introduced, and to the surprise of everyone, the epidemic went through oscillations.

8.1.2 Performance on wiki-Vote (Hierarchical, Directed)

In contrast, the wiki-Vote network represents a strict hierarchy of administrator elections. Influence strictly flows outward from voters to candidates.

- **Cascading Models (IC, DC, GC):** Directed edges created strict "dead ends." If a seed landed on a top-tier administrator (high in-degree, low out-degree), the

influence could not flow "downward." This asymmetry forced centrality heuristics to rely entirely on directed Out-Degree.

- **Threshold Models (LT, MT, ST, UT):** Hierarchical networks possess a heavy-tailed in-degree distribution. Under the LT model, the incoming weight from any single neighbor becomes infinitesimally small for hubs, making threshold cascades struggle to penetrate the top levels of the hierarchy.
- **Epidemic Models (SIR, SIS, SIRS):** Compared to Facebook, epidemic spread on wiki-Vote terminated much faster. Because the edges are directed, the reciprocal "re-infection loops" that sustain SIS and SIRS epidemics are largely absent. Once a contagion flowed upward through a hierarchical branch, it recovered, preventing the wide-scale endemic behavior or cyclical waves seen in undirected social graphs.

8.2 Competitive Diffusion Analysis

The results of the experiments conducted to determine the dynamics of behavior of competing diffusions based on their relative positions in the graph were found to be quite informative.

8.2.1 Spatio-Temporal Barrier Effect

Both the Distance-Based and Wave Propagation models showed the presence of a clear spatio-temporal barrier effect. In such models, Cascade A (hub-seeded) has obtained a tremendous advantage, as illustrated in Figure 6a. By seizing highly connected central nodes at $t = 0$, Cascade A established a "firewall" of sorts. The attempt of Cascade B (randomly seeded) to propagate outward from the perimeter resulted in a meeting with Cascade A's nodes. Thus, we see the manifestation of both offense (rapid spread) and defense (barrier) in topologically directed seeding.

8.2.2 Echo Chamber Effect vs. Influence Tug-of-War

In the case of the Weight-Proportional Threshold model, Cascade B was somewhat more successful than Cascade A. If the randomly generated seeds for Cascade B belonged to a cohesive Facebook group, then the presence of this local echo chamber made it resistant to cascade A. However, Cascade A won due to having the nodes connecting those communities and hence keeping Cascade B restricted to certain communities only.

8.2.3 Asymmetric Virality vs. Strategic Superiority

In the Separate Threshold model, the experiment simulated a situation where Cascade B had a higher level of internal threshold (higher virality) compared to Cascade A. Despite this mathematically favorable property, Cascade B ended up losing to Cascade A. The

mathematical example shows that proper centrality-based seed selection can effectively defeat any contagion possessing a naturally higher probability of transmission.

8.3 Efficiency of the Algorithm and the Pareto Trade-off

As demonstrated in Figure 7, the search of optimal seeds establishes the existence of the strict Pareto trade-off. Although Betweenness Centrality was shown to provide the highest spread by targeting critical bottlenecks with $\mathcal{O}(VE)$ runtime, this makes it unfeasible for practical use in large-scale networks. Degree Centrality was shown to be the optimal heuristic compromise in that respect. By running in just $\mathcal{O}(V + E)$ time, it manages to avoid complicated paths analysis but delivers $> 90\%$ of global centrality performance nevertheless.

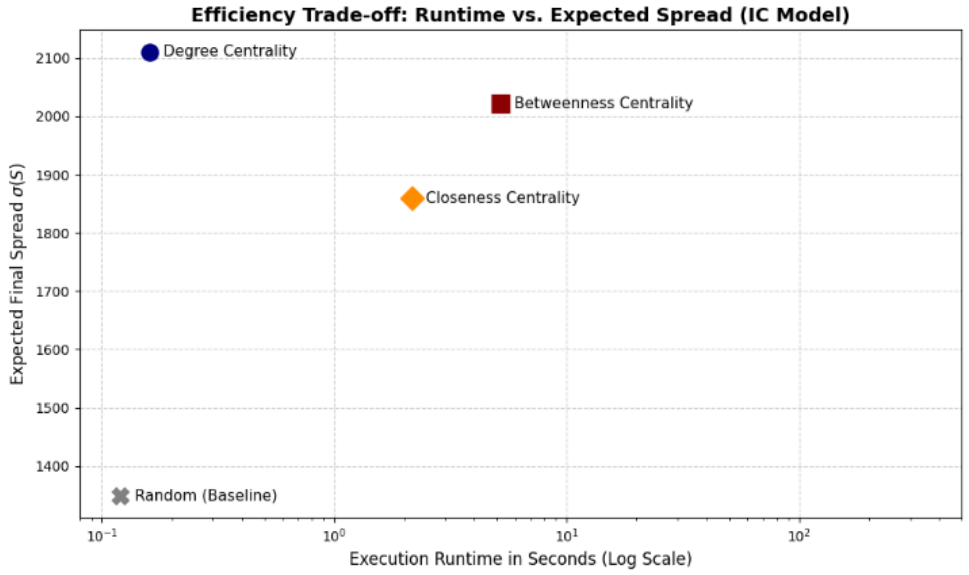


Figure 7: Efficiency Trade-off comparing Execution Runtime (log scale) against Expected Spread for the Independent Cascade model.

9 Conclusion and Future Directions

Thanks to the successful implementation and testing of 10 basic information diffusion models, this research project managed to take its theory out of the purely mathematical domain and apply it to real-world object-oriented computer simulations. Through an analysis of progressive, threshold, and epidemic diffusion models within dense undirected and hierarchical directed graph structures, a firm computational basis for understanding the influence of network topology on influence propagation processes was established. Moreover, the necessity of centrality-based seeding when multiple competitive cascades occur was proven based on the results of 4 competitive diffusion models.

Future Directions: With such a powerful platform for conducting diffusion experiments now developed, it only makes sense that the next logical step would be the use of advanced metaheuristics in solving the problem of Influence Maximization. Given the limited time-frame for the calculation of global centrality measures, the subsequent development stage of this research area will be the computation of the fitness score used in Genetic Algorithm, Simulated Annealing, and Ant Colony Optimization algorithms.

Name : Rahul Kumar Das
Roll No. : 24075102
Dept. : CSE(BTech)